AI-K8sGuard: Project Progress Report

Implementation, Prototype Status, Testing and Validation of an Explainable AI-Assisted Framework for Kubernetes Security Assessment and Failure Diagnosis (Hybrid Rule-Based and Local LLM Approach)

Summary of Progress

AI-K8sGuard assesses the security of Kubernetes workloads and helps diagnose operational failures without sending cluster data to a paid external AI service. Its central design rule is a strict separation of duties: a **deterministic rule engine** decides what is a security finding and how serious it is, while a **locally hosted LLM** (Qwen2.5 3B served through Ollama) only explains verified evidence.

The deterministic half of the system is built and working end to end: a Deployment YAML is parsed, evaluated against the security rules, scored, enriched with an evidence package, stored in MongoDB and displayed in a React dashboard, which also reads live state from a local Minikube cluster. The LLM half is wired into the interface but is currently **deferred** (Ollama/Qwen2.5 3B is not yet enabled), so no LLM-generated explanation, failure diagnosis or comparative experiment has been produced yet. This report therefore states clearly which results are measured, which are observed only qualitatively, and which are still pending.

Table 1. Prototype status at a glance

Area	Status	Basis
YAML parsing, rule engine, severity-weighted risk score	**Completed**	Privileged-container manifest: 3 findings, 24 points, CRITICAL; secure baseline: 0 findings (Figs. 7, 11)
Live Kubernetes read-only monitoring dashboard	**Completed**	1/1 node, 7/7 pods, 1/1 deployment, 2 services shown; agrees with kubectl (Figs. 3, 5, 16)
Report generation and MongoDB persistence	**Completed**	16 stored security-analysis records listed in the Reports view (Fig. 14)
Evidence builder (finding + YAML + status + events + logs)	**Partial**	Structure present; runtime fields empty for workloads not deployed in the cluster (Fig. 8)
Failure-scenario diagnosis (CrashLoopBackOff, etc.)	**Partial**	Interface built; no failing workload deployed yet, so no diagnosis run (Fig. 13)
Local LLM interpretation (Ollama + Qwen2.5 3B)	**Deferred**	Interface shows 'Deferred'; no model output generated (Fig. 9)
Evaluation: rule-only vs LLM-only vs hybrid	**Planned**	Harness and scenario set not yet built; no accuracy/F1 figures exist

1. Methodology and Implementation

1.1 Architecture and pipeline

The system follows a layered client–server design and a one-directional pipeline in which data becomes progressively more structured and more trusted: *YAML/cluster input -> analysis -> security rules -> risk assessment -> evidence builder -> local LLM -> explainable report*. Only verified findings and scoped evidence ever reach the LLM; unrestricted cluster data is never forwarded, and the LLM has no write access to the cluster.

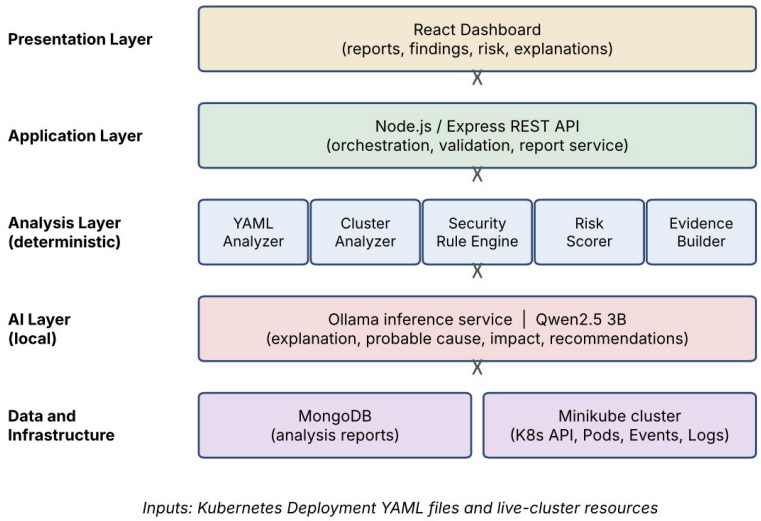


Fig. 1. *Layered architecture of AI-K8sGuard (presentation, application, deterministic analysis, local AI, data/infrastructure).*

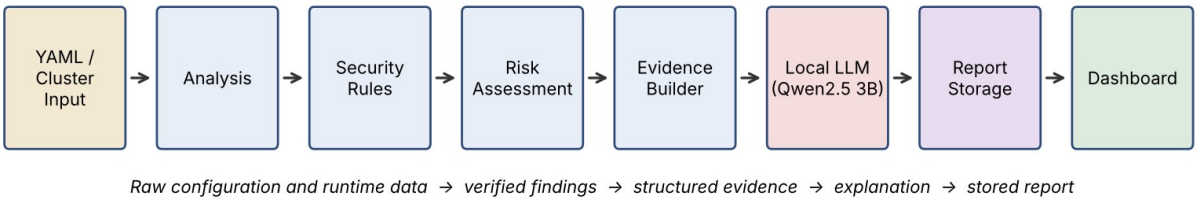


Fig. 2. *End-to-end data flow: raw configuration and runtime data -> verified findings -> structured evidence -> explanation -> stored report.*

1.2 Algorithms

Deterministic rule evaluation. Every rule is applied to every container of the normalized workload; a finding $F_i = \{r_i, e_i, s_i, w_i\}$ (rule, evidence, severity, weight) is emitted when the condition holds. The same input always yields the same findings.

```
Algorithm 1: Rule evaluation
Input : normalized workload configuration W, rule set RULES
Output: findings F
for each rule r in RULES:
    for each container c in W:
        if r.condition(c):
            F.append({rule: r.id, evidence: extract_field_evidence(c, r.field),
                      severity: r.severity, weight: WEIGHT[r.severity]})
return F
```

Severity-weighted risk score. R = sum of w_i over all findings of a workload, with the initial weights Critical = 10, High = 7, Medium = 4, Low = 2. The prototype is consistent with these weights: the privileged-container finding (Critical) contributes 10 points. The score is for prioritization and comparison of controlled configurations, not a universal security measure.

Evidence construction. For each finding or failure the Evidence Builder assembles a bounded package E = F + Y + S + V + L (finding, YAML evidence, workload status, events, logs). The prototype's Runtime Evidence panel shows exactly these fields: finding, rule, Pod status, Deployment status, events, logs and configuration (Fig. 8).

Failure-scenario detection. Four scenarios are targeted, each recognised from observable Kubernetes state: CrashLoopBackOff (waiting reason, restart count, logs, events), ImagePullBackOff (pull-failure events, image reference), missing Secret/ConfigMap (configuration-error status, events) and readiness failure (Pod Ready condition, probe events).

Table 2. Security rules and severities (initial, proposed in the design)

Rule	Condition checked	Severity	Implementation status
Privileged container	securityContext.privileged = true	Critical	Verified (rule K8S-001, 10 points, Fig. 8)
Root execution	runs as UID 0 / does not require non-root	High	Implemented per design; not individually verified
Privilege escalation	allowPrivilegeEscalation permitted	High	Implemented per design; not individually verified
Host-level access	hostNetwork / hostPID / hostIPC / hostPath	High	Implemented per design; not individually verified
Excessive capabilities	dangerous capabilities added	High	Implemented per design; not individually verified
Missing resource limits	CPU/memory limits not set	Medium	Implemented per design; not individually verified
Mutable image tag	latest or no explicit tag	Medium	Implemented per design; not individually verified
Missing probes	no readiness/liveness probe	Low	Implemented per design; not individually verified

Note. The test manifest returned three findings (24 points) but only the first (K8S-001) is visible in the captured screenshots. The remaining 14 points are consistent with two High-weight findings (2 x 7), but this must be confirmed from the stored report before it is quoted. The prototype labels rules K8S-0xx; the mapping of the other rules to the design's R2–R8 numbering is to be confirmed.

1.3 Models, frameworks and development tools

Table 3. Technology stack

Layer	Technology	Role	Status
Orchestration	Kubernetes on Minikube, kubectl	Local single-node cluster supplying workloads, events, logs	**Verified**
Local LLM runtime	Ollama	Serves the model through a local HTTP API	**Pending**
Language model	Qwen2.5 3B	Explanation and failure diagnosis over structured evidence	**Pending**
Backend	Node.js with Express (JavaScript)	REST API, analyzers, rule engine, evidence builder, LLM client	**Verified**
Frontend	React	Dashboard: YAML editor, analysis, diagnosis, reports	**Verified**
Database	MongoDB 7 (container image mongo:7)	Stores one report document per analysis	**Verified**
Containers / host	Docker Desktop, VS Code, Git, PowerShell	Local runtime and development environment	**Verified**

Qwen2.5 3B was chosen because it runs on modest hardware; its lower reasoning capability compared with larger models is an accepted trade-off and the reason larger models are listed as future work. The prompt (planned) instructs the model to answer only from the supplied evidence, to say when evidence is insufficient, and to return a structured output (explanation, probable cause, impact, severity interpretation, recommendations) at a low sampling temperature to reduce run-to-run variability. The model is never asked to decide whether a configuration is secure.

1.4 Database design

MongoDB holds one report document per analysis run, so a complete report (findings, evidence, LLM output) can be retrieved with one query. Report identifiers of the form SEC-<timestamp>-<suffix> are visible in the Reports view, and the dashboard lists 24-character hexadecimal document identifiers (Figs. 4, 14).

Table 4. Report document structure (field names proposed in the design)

Field	Type	Description
_id, createdAt	ObjectId, date	Identifier and creation time
source / target / mode	string / object / string	yaml or cluster; workload name, namespace, kind; rule-only, llm-only or hybrid
yamlSnapshot	string	Analyzed configuration
findings[]	array of objects	ruleId, severity, weight, evidence per finding
riskScore	number	Cumulative risk R
evidence / failure	object	Status, events, logs and configuration; detected failure scenario
llm	object	model, explanation, cause, impact, severityInterpretation, recommendations[], latencyMs

1.5 Hardware components

The prototype runs on a single Windows machine (VS Code and PowerShell, project folder F:\AI-Ku) that hosts Docker Desktop, the Minikube cluster, MongoDB and the web application; Ollama will run on the same machine. No dedicated sensors or special hardware are involved. Docker Desktop reports **4 CPUs and 3.68 GB of memory** available to its engine (Fig. 15). The host processor model, total RAM and GPU were not captured and must be recorded before latency figures are reported.

1.6 APIs and integration procedure

Table 5. APIs used

API	Operation	Purpose	Status
Kubernetes API (read-only)	List/read Pods, Deployments, Services, Events; read Pod logs	Runtime evidence and dashboard	Verified
Backend REST API	YAML analysis, cluster analysis, diagnosis, report list/fetch (endpoint names proposed)	Serves the React dashboard	Verified
Ollama HTTP API	Generate/chat with qwen2.5:3b	Local LLM inference	Pending
MongoDB driver	Insert/query report documents	Persistence	Verified

Table 6. Integration procedure and current status

#	Step	Status
1	Start Minikube and confirm access with kubectl (context minikube-recovery)	Completed
2	Deploy controlled test workloads (secure, misconfigured, failure-inducing)	Partial
3	Start MongoDB and confirm the backend can connect	Completed
4	Install Ollama, pull qwen2.5:3b, confirm a test prompt returns	Pending
5–6	Start Express backend and React dashboard and connect them	Completed
7	Run YAML analysis; confirm findings and risk score	Completed
8	Run live-cluster analysis and failure diagnosis; confirm LLM output is parsed	Partial
9	Confirm the report is stored in MongoDB and shown on the dashboard	Completed
10	Run the evaluation harness across the three configurations	Planned

2. Prototype Development

2.1 Functional modules

Table 7. Module status (design modules M1–M10)

ID	Module	Status	Evidence / remark
M1	YAML Security Analyzer	**Completed**	Parses Deployment YAML; handles valid, secure and malformed input (Figs. 10–12)
M2	Kubernetes Cluster Analyzer (read-only)	**Completed**	Live nodes, pods, deployments, services shown (Figs. 3, 5). Events/log collection not yet captured
M3	Failure Scenario Detector	**Partial**	Selection UI present; no failing Pod exists to detect (Fig. 13)
M4	Security Rule Engine	**Completed**	K8S-001 verified with evidence text; other rules pending individual tests
M5	Risk Assessment Module	**Completed**	24 points for 3 findings; 0 for secure baseline
M6	Evidence Builder	**Partial**	Works for static manifests; runtime fields empty when workload is not deployed
M7	Local LLM Service	**Deferred**	Graceful 'deferred' state implemented; no inference yet
M8	Report Service + MongoDB	**Completed**	16 stored records (Fig. 14)
M9	Web Dashboard	**Completed**	Five views: Dashboard, Security Analysis, YAML Analysis, Failure Diagnosis, Reports
M10	Evaluation Harness	**Planned**	Not started

Six of ten modules are operational, three are partial or deferred, and one is planned. Completion is stated by module status rather than as a percentage, because the remaining effort (LLM enablement and the evaluation) is not comparable in size to the finished modules.

2.2 Implemented features and working-component evidence

Dashboard. A live overview shows Kubernetes connection state, node/pod/deployment/service counts, the latest analysis and a list of recent analyses. Header indicators show Kubernetes *Connected*, Ollama *Deferred* and the environment as *Local*; the sidebar shows MongoDB *Connected*. The application is served locally at localhost:8443.

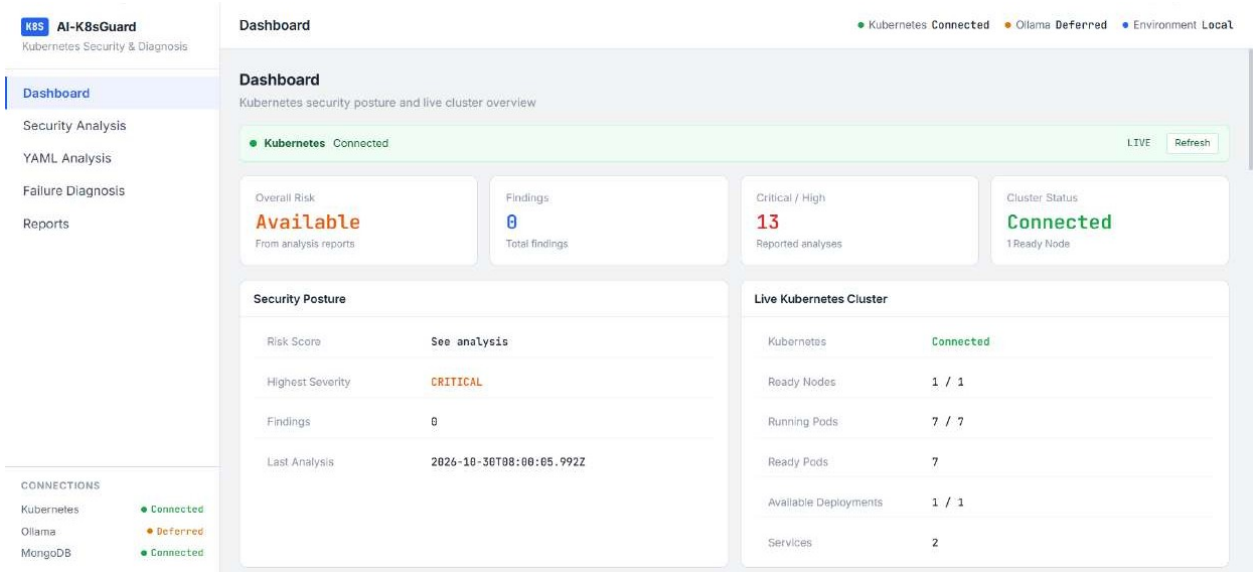


Fig. 3. *Dashboard: cluster status (1 ready node, 7/7 running pods, 1/1 deployment, 2 services), 13 critical/high reported analyses, and connection status of Kubernetes, Ollama and MongoDB.*

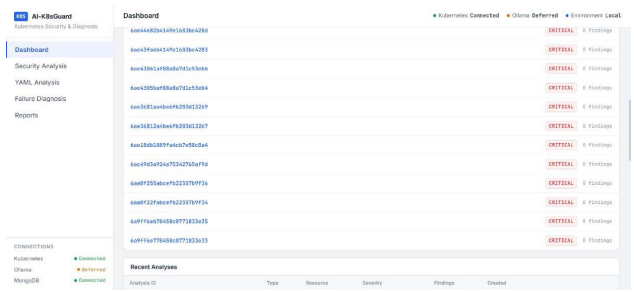


Fig. 4. Recent analyses listed with their severity and finding count.

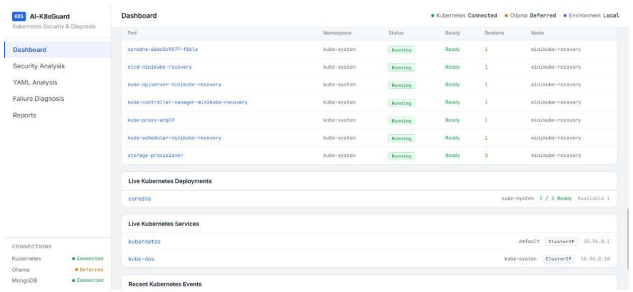


Fig. 5. Live pod table (kube-system, all Running/Ready), deployment and service lists.

Security Analysis. The view states that security decisions are authoritative from the rule engine and that LLM output is supplementary and cannot override findings. For the test deployment privileged-container-test (namespace ai-k8sguard) it reports a risk score of 24 points, highest severity CRITICAL and 3 findings. Each finding lists rule ID, type, severity, risk points, resource, namespace, evidence and source. The first, K8S-001, is a privileged container (10 points) with the evidence text *Container "nginx" has securityContext.privileged=true*. The Runtime Evidence panel reports no Pod, no Deployment, 0 events and no logs because this workload was analysed from YAML and not deployed.

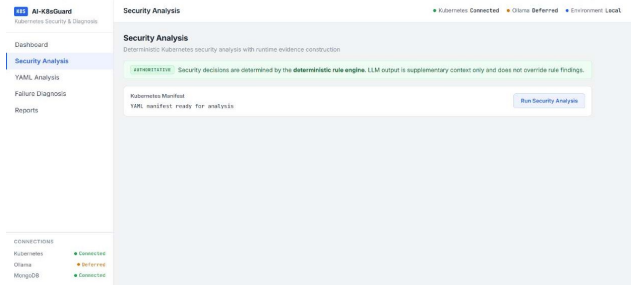


Fig. 6. Security Analysis view before a run.

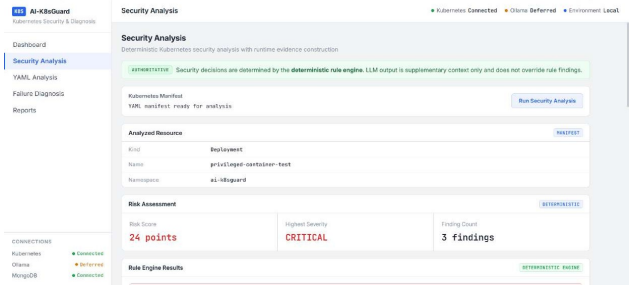


Fig. 7. Result summary: 24 points, CRITICAL, 3 findings.

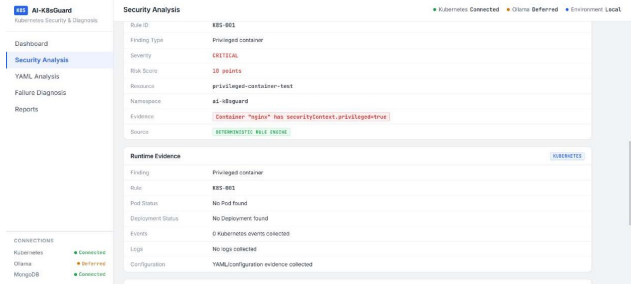


Fig. 8. Finding K8S-001 and the Runtime Evidence panel.

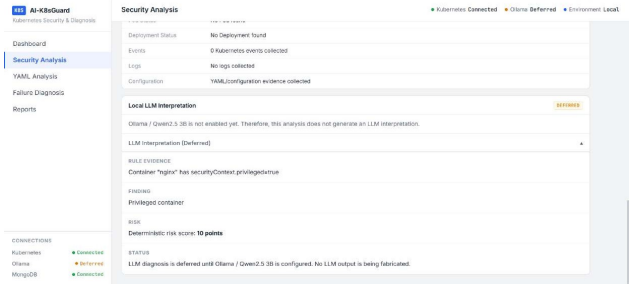


Fig. 9. Local LLM Interpretation panel: DEFERRED; no LLM output is fabricated.

YAML Analysis. An editor with a five-step progress indicator (YAML, Parser, Security Rules, Findings, Risk Assessment) and buttons Load Example, Secure Baseline, Invalid YAML, Clear and Analyze. The secure baseline completes all five steps with 0 findings, severity NONE, 0 points and the message 'No security violations detected'; the malformed manifest returns a controlled ANALYSIS_ERROR with a Back button instead of a crash.

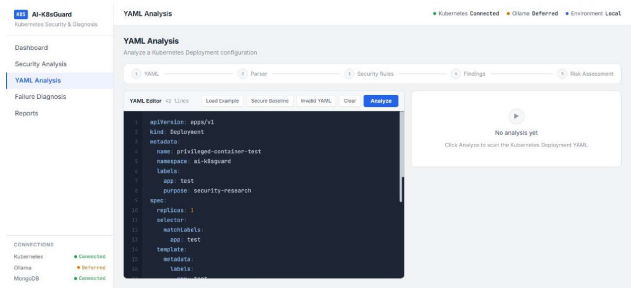


Fig. 10. YAML editor loaded with the privileged-container test manifest (42 lines), before analysis.

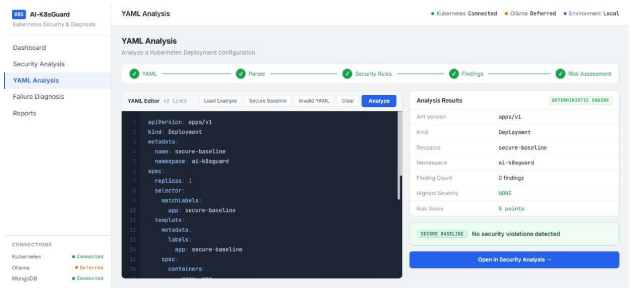


Fig. 11. Secure baseline (43 lines): all steps pass, 0 findings, 0 points.

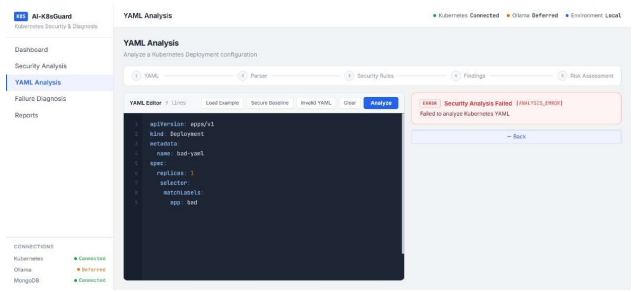


Fig. 12. Malformed manifest (9 lines): controlled error ANALYSIS_ERROR.

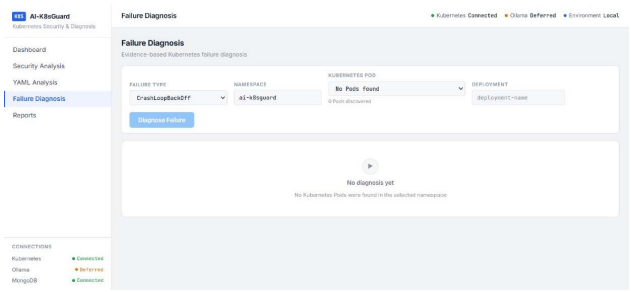


Fig. 13. Failure Diagnosis view: failure type, namespace, Pod and deployment selectors; no Pods found in ai-k8sguard.

Reports. The Reports view lists stored analyses with ID, type, severity, risk score and creation time, with a detail panel on the right. Sixteen records are stored; the visible rows show one NONE (risk 0) and four CRITICAL (risk 24) analyses.

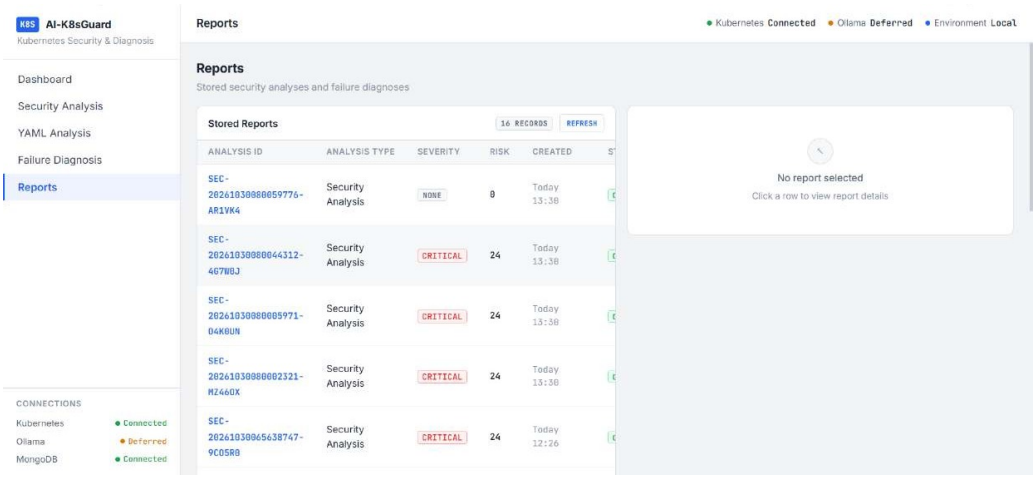


Fig. 14. Reports view: 16 stored records of type Security Analysis with severity and risk score.

2.3 Implementation milestones reached

- Local Kubernetes cluster running and queryable (kubectl and the application agree on pod state).
- Backend, frontend and MongoDB connected; analyses persisted and listed.
- Deterministic rule engine producing findings with evidence text and a severity-weighted risk score.
- Secure-baseline and malformed-input behaviour handled correctly.
- Explicit, honest 'LLM deferred' state: the UI reports that no LLM output is generated rather than showing placeholder text.

3. Dataset and Experimental Setup

3.1 Data sources and current dataset

The project does not use a pre-existing training dataset. The LLM is used for inference only, and the experiments use **controlled, author-created Kubernetes scenarios** with known ground truth. Two data sources feed the system: (i) Deployment YAML manifests and (ii) read-only runtime information (status, events, logs) from the local Minikube cluster.

Table 8. Data used so far

Item	Size / content	Purpose
privileged-container-test (Deployment, ns ai-k8sguard)	42-line manifest, nginx container with privileged=true	Positive security test
secure-baseline (Deployment, ns ai-k8sguard)	43-line manifest, no violations	Negative (clean) control
Malformed Deployment	9-line incomplete manifest	Error-handling test
Live cluster snapshot	1 node, 7 pods (kube-system), 1 deployment (coredns), 2 services	Runtime evidence, dashboard
Stored reports	16 security-analysis records in MongoDB	Persistence and repeatability

Preprocessing. YAML text is parsed into a normalized workload object (containers, security context, resources, probes, image, host settings, capabilities). For LLM use, the Evidence Builder selects only the YAML fields relevant to the finding and bounds the number of event and log lines, keeping the prompt within the limited context of a small model.

Planned evaluation dataset (proposed). At least 20 security manifests (one clean baseline, one single-rule misconfiguration per rule, and combined cases) and at least 12 failure runs (4 scenarios x 3 repetitions), each with a written ground-truth label, evaluated in all three configurations. These sizes are targets to be confirmed, not completed work.

3.2 Hardware and software environment

Table 9. Environment as captured

Component	Observed configuration
Host / OS	Windows workstation; PowerShell terminal; VS Code (git branch main); project folder F:\AI-Ku
Docker Desktop	Engine running; 4 CPUs and 3.68 GB memory available; disk 8.73 GB used; reports 'outdated version'
Kubernetes	Minikube, single node minikube-recovery; kubectl context minikube-recovery; API server exposed on a local port
Database	MongoDB container (mongo:7), port 27017
Application	React frontend and Node.js/Express backend, served at localhost:8443
LLM	Ollama + Qwen2.5 3B planned; not enabled
Not yet recorded	Host CPU model, total RAM, GPU (if any), Ollama and model version, quantization

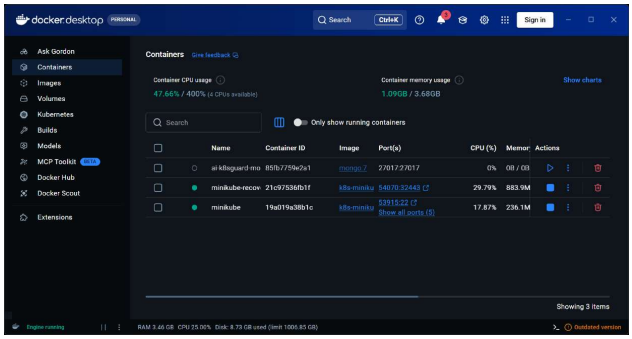


Fig. 15. Docker Desktop: MongoDB (mongo:7) and two Minikube containers; 47.66% of 400% CPU and 1.09 GB of 3.68 GB memory in use. The MongoDB container appears stopped at the instant of this capture, while the application captures show MongoDB as Connected.

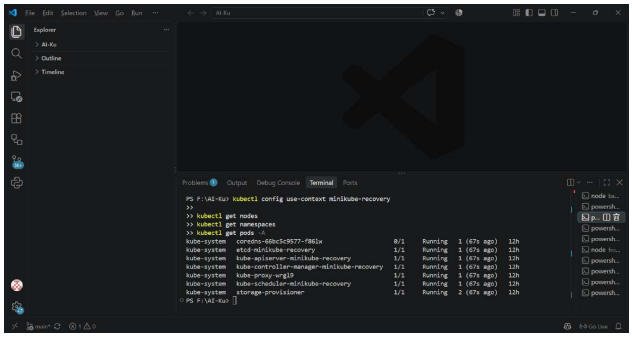


Fig. 16. VS Code terminal: kubectl context minikube-recovery; all seven kube-system pods Running.

3.3 Parameter settings and evaluation methodology

Table 10. Parameters

Parameter	Value	Status
Severity weights	Critical 10, High 7, Medium 4, Low 2	Info
Risk aggregation	Additive sum R = sum of w_i	Info
LLM model / runtime	qwen2.5:3b via Ollama	Pending
Sampling temperature, context length, log/event line limits	Low temperature planned; values to be fixed and recorded	Pending
Repetitions per scenario	Planned: 3 (for latency and consistency)	Planned

Evaluation methodology. Three configurations are compared on the same scenarios: *rule-only* (deterministic rules only), *LLM-only* (the LLM receives the Kubernetes information without the decision layer) and *hybrid* (rules establish findings, the LLM explains them). Each result is compared with the ground-truth label. Security detection and diagnosis are scored with precision, recall and F1 (precision = TP/(TP+FP), recall = TP/(TP+FN), F1 = 2PR/(P+R)); evidence grounding and explainability use a written scoring rubric; latency is the time from request to completed report; resource use is CPU and memory during inference.

4. Testing and Validation

4.1 Test cases, expected versus observed outcomes

Tests so far are functional tests executed through the user interface and the terminal, evidenced by the screenshots. They are not yet an automated test suite.

Table 11. Executed and pending test cases

ID	Scenario	Expected	Observed	Result
TC-01	Analyze secure baseline YAML (Fig. 11)	0 findings, no risk	0 findings, NONE, 0 points; 'No security violations detected'	**Pass**
TC-02	Analyze privileged-container manifest (Figs. 7, 8)	Critical finding for privileged container	K8S-001 CRITICAL, 10 points, correct evidence text; 3 findings, 24 points in total	**Pass**
TC-03	Analyze malformed YAML (Fig. 12)	Controlled error, no crash	ANALYSIS_ERROR message and Back button	**Pass**
TC-04	Analyze manifest not deployed in cluster (Fig. 8)	Runtime evidence reported as absent	'No Pod found', 0 events, no logs; configuration evidence collected	**Pass**
TC-05	Live cluster overview versus kubectl (Figs. 3, 5, 16)	Same pods and states	7 pods Running in both; kubectl capture shows coredns 0/1 immediately after a restart	**Pass**
TC-06	LLM unavailable (Fig. 9)	No fabricated output; clear status	'LLM diagnosis is deferred ... No LLM output is being fabricated'	**Pass**
TC-07	Failure Diagnosis with no Pods (Fig. 13)	Action blocked, message shown	Diagnose button disabled; 'No Kubernetes Pods were found'	**Pass**
TC-08	Report persistence (Fig. 14)	Analyses stored and listed	16 records listed with ID, severity, risk, time. Opening a record not yet captured	**Partial**
TC-09	Repeatability of risk score	Identical input gives identical score	Four stored records show identical severity and score (CRITICAL, 24); same manifest not confirmed from capture	**Partial**
TC-10	LLM explanation of a finding	Structured explanation parsed and stored	Not run (Ollama deferred)	**Pending**
TC-11	CrashLoopBackOff, ImagePullBackOff, missing Secret/ConfigMap, readiness failure	Correct scenario detected and diagnosed	Not run (no failing workloads deployed)	**Pending**
TC-12	Rules R2–R8 individually	Each rule fires on its own violation only	Not run	**Pending**

All results above are functional checks on a small number of inputs (n = 3 manifests). They show that the implemented path behaves as designed; they are not an accuracy estimate.

4.2 Identified defects, debugging and corrective measures

Table 12. Defect and observation log

ID	Observation	Likely cause (hypothesis)	Corrective action	Status
D-1	Dashboard shows 'Findings 0' and '0 findings' for CRITICAL analyses, while the Security Analysis view reports 3 findings (Figs. 3, 4 vs 7)	Dashboard reads a different field from the stored report than the analysis view	Align the dashboard aggregation with the stored report schema; add a test comparing both views	Open
D-2	Overall Risk shows the word 'Available' and Risk Score 'See analysis' instead of a number (Fig. 3)	Aggregate score not yet computed for the dashboard	Show latest and maximum risk score numerically	Open
D-3	Malformed YAML returns only a generic message 'Failed to analyze Kubernetes YAML' (Fig. 12)	Parser errors are not passed to the interface	Return line/field-level parse or schema errors	Open
D-4	Control-plane pods show restarts and the cluster profile is named minikube-recovery (Figs. 5, 16)	Cluster instability during development required recovery	Cluster recovered and in use; monitor restarts and document the recovery steps	In progress
D-5	MongoDB container shown stopped in Docker Desktop (Fig. 15)	Container not set to restart automatically, or stopped between sessions	Set a restart policy and verify the connection state before each run	Open
D-6	Docker Desktop 'outdated version'; one problem in the VS Code Problems panel (Figs. 15, 16)	Not yet reviewed	Update Docker Desktop; review the editor problem	Open

Debugging so far has relied on comparing the application's view with kubectl output (TC-05) and on stepping through the five-stage YAML pipeline indicator, which shows at which stage an analysis stops. Causes in the table are working hypotheses and have not yet been confirmed in code.

5. Quantitative Results and Performance Analysis

Scope of this section. The accuracy, precision, recall, F1-score and latency evaluation depends on the evaluation harness and the LLM, which are not yet in place. Only values that can be read directly from the captured evidence are reported below, together with their conditions. No estimated or placeholder figures are included.

5.1 Measured results

Table 13. Measured and observed values (single machine, LLM not running)

Measure	Value	Conditions / source
Risk score, privileged-container-test	24 points (3 findings, CRITICAL)	YAML analysis; Fig. 7
Risk score contribution, K8S-001 (Critical)	10 points	Matches proposed Critical weight; Fig. 8
Risk score, secure baseline	0 points (0 findings, NONE)	YAML analysis; Fig. 11
Functional outcome agreement	3 of 3 test inputs behaved as expected	Secure, misconfigured and malformed manifests; n = 3, not an accuracy estimate
Stored reports	16 records; 13 critical/high reported analyses	Reports and Dashboard views; Figs. 3, 14
Live cluster state	1/1 nodes, 7/7 pods Ready, 1/1 deployments, 2 services	Read-only Kubernetes API; Fig. 3
Pod restarts	1 on each control-plane pod; storage-provisioner higher (2 in the kubectl capture, 3 on the dashboard)	Observed at different moments; Figs. 5, 16

5.2 Resource utilization

Table 14. Docker Desktop utilization at capture (Minikube and MongoDB present, Ollama not running)

Container	CPU	Memory	Remark
minikube-recovery (active cluster)	29.79%	883.9 MB	Single-node control plane and workloads
minikube	17.87%	236.1 MB	Second Minikube container
MongoDB (mongo:7)	0%	0 B	Container stopped at capture
Total (Docker engine)	47.66% of 400% (4 CPUs)	1.09 GB of 3.68 GB	Host: RAM 3.46 GB in use, CPU 25%, disk 8.73 GB used

This is a single instantaneous reading, not an average. It indicates about 2.6 GB of memory headroom inside the Docker limit before the LLM and a running MongoDB are added. Qwen2.5 3B is a small quantized model, but its actual footprint, and whether Ollama runs inside or outside Docker, must be measured; memory is expected to be the main constraint on this machine.

5.3 Metrics pending the evaluation

Table 15. Planned performance indicators

Indicator	Definition	Status
Security-detection accuracy, precision, recall, F1	Detected findings versus ground truth, per configuration	**Pending**
Failure-diagnosis accuracy	Identified cause matches the injected failure	**Pending**
Evidence grounding, explainability	Rubric score of whether explanation claims are supported by supplied evidence	**Pending**
End-to-end latency, execution time	Request to completed report; rule-only analysis versus LLM step, over repeated runs (mean and spread)	**Pending**
Throughput, error rate	Analyses per minute; failed or unparsable analyses and LLM outputs	**Pending**
CPU / memory during inference	Peak and mean while Qwen2.5 3B is generating	**Pending**

6. Integration and Verification

6.1 Module and interface integration

Table 16. Interfaces between modules (design interfaces I1–I10)

ID	Interface	Status	Evidence
I1	Dashboard -> Backend (HTTP REST)	**Verified**	Analyze, run and report actions return results in the UI
I2	Backend -> M1 / M2 (in-process)	**Verified**	YAML analysis and live cluster data both work
I3	M2 -> Kubernetes API (HTTPS, read-only)	**Verified**	Live pods, deployments, services, node count; matches kubectl
I4	M1 / M2 -> M4 (normalized config)	**Verified**	Findings generated from parsed YAML
I5	M4 -> M5 (findings with weight)	**Verified**	24-point score from 3 findings
I6	M4 / M3 / M1 / M2 -> M6 (evidence)	**Partial**	Static evidence present; runtime fields empty without a deployed workload
I7	M6 -> M7 -> Ollama (HTTP)	**Pending**	Ollama deferred
I8	M7 -> M8 (LLM output)	**Partial**	Deferred-state path shown; model-output path untested
I9	M8 <-> MongoDB	**Verified**	16 records persisted
I10	Backend -> Dashboard (stored reports)	**Verified**	Reports list populated; dashboard counts inconsistent (D-1)

6.2 System-level verification

- **Verified:** the static pipeline YAML input -> parser -> rules -> risk score -> evidence -> report -> MongoDB -> dashboard, for valid, secure and malformed inputs.
- **Verified:** read-only live-cluster integration; dashboard counts agree with kubectl output.
- **Verified:** the safety property that the system does not fabricate LLM output when the LLM is unavailable, and that rule findings stay authoritative.
- **Not yet verified:** the live pipeline cluster -> failure detector -> evidence -> LLM -> report, and the hybrid configuration as a whole.
- **Not yet verified:** behaviour under repeated runs, restarts of the cluster or database, and concurrent requests.

7. Technical Challenges and Limitations

Table 17. Challenges, limitations and strategies

Issue	Type	Impact	Adopted / proposed strategy
Ollama and Qwen2.5 3B not yet enabled	Unresolved	No LLM explanation, diagnosis or LLM-only/hybrid experiment can be run	Install Ollama, pull qwen2.5:3b, verify a test prompt, then connect to M7 (highest priority)
Limited machine resources (4 CPUs, 3.68 GB Docker memory)	Constraint	Minikube, MongoDB and the model compete for memory; inference latency may be high	Measure model footprint; run Ollama on the host outside Docker; close unused containers; use CPU-only latency as the reported condition
Cluster instability / recovery of Minikube	Environment	Lost time; restarts affect reproducibility	Recovered profile in use; document recovery steps; scripted setup of test workloads
No failing workloads deployed	Unresolved	Failure detector and diagnosis untested; runtime evidence empty	Create manifests for the four failure scenarios and deploy them in ai-k8sguard
Rule coverage limited to eight Deployment-level checks	Design limitation	RBAC, NetworkPolicy, Secrets and image-vulnerability issues are not detected	State scope in the paper; extend the rule set as future work
Additive risk score is uncalibrated and unbounded	Design limitation	Scores are comparable only between controlled cases	Present as a prioritization aid; test alternative weights during evaluation
Small-model reliability	Risk	Explanations may be plausible but unsupported by evidence	Evidence-only prompts, low temperature, grounding rubric, and the rule decision layer stays authoritative
Dashboard inconsistencies and generic errors (D-1 to D-3)	Defect	Misleading summary figures	Fix field mapping; improve error detail; add regression tests

Failed experiments. No formal experiment has failed, because the experimental phase has not started. The difficulties encountered so far are environmental (cluster recovery) and are recorded as defects above. **Performance bottlenecks** have not been measured; the likely bottleneck is LLM inference on CPU within the available memory, to be confirmed by measurement.

8. Milestone Achievement and Future Work

8.1 Status of activities

Table 18. Completed, partial, in-progress and planned work

Category	Activities
Completed	Architecture and module design; local Minikube cluster; MongoDB; React dashboard with five views; YAML parsing; rule engine with K8S-001 verified; severity-weighted risk scoring; read-only live-cluster monitoring; report storage and listing; secure-baseline and malformed-input handling; transparent 'LLM deferred' behaviour.
Partially implemented	Evidence builder (runtime fields empty without a deployed workload); failure-diagnosis interface (no scenarios deployed); report detail view (not yet captured); verification of rules R2–R8 (only K8S-001 evidenced).
In progress	Enabling Ollama with Qwen2.5 3B (interface currently shows Deferred); stabilising the Minikube environment; resolving dashboard inconsistencies D-1 and D-2.
Planned	Failure-scenario workloads; LLM prompt and output parser; evaluation harness (M10); scenario dataset with ground truth; three-configuration experiments; grounding and explainability scoring; latency and resource measurement; automated tests; final paper results.

8.2 Remaining work

Table 19. Remaining development, testing, optimization and validation

Step	Activity	Type
1	Install Ollama, pull qwen2.5:3b, verify a test prompt; record host hardware and versions	Development
2	Implement prompt template and structured-output parser in M7; store LLM output and latency in the report	Development
3	Write and deploy manifests for CrashLoopBackOff, ImagePullBackOff, missing Secret/ConfigMap and readiness failure	Development
4	Individual tests for rules R2–R8; unit tests for rule engine, scoring and evidence builder	Testing
5	Fix D-1 to D-6; add field-level YAML error messages	Development
6	Build the scenario dataset with ground-truth labels and the evaluation harness (M10)	Development
7	Run rule-only, LLM-only and hybrid experiments with repetitions; compute precision, recall, F1, diagnosis accuracy	Validation
8	Score evidence grounding and explainability; measure latency, throughput, CPU and memory	Validation
9	Optimization: evidence size limits, prompt length, model quantization, and comparison with a larger local model if resources allow	Optimization
10	Consolidate results, limitations and conclusions in the final paper	Documentation

Conclusion. The deterministic core of AI-K8sGuard, its live-cluster view and its reporting are working and evidenced. The decisive remaining steps are enabling the local LLM and running the controlled evaluation; until then no claim about LLM benefit, accuracy or latency can be made.